NANYANG TECHNOLOGICAL UNIVERSITY

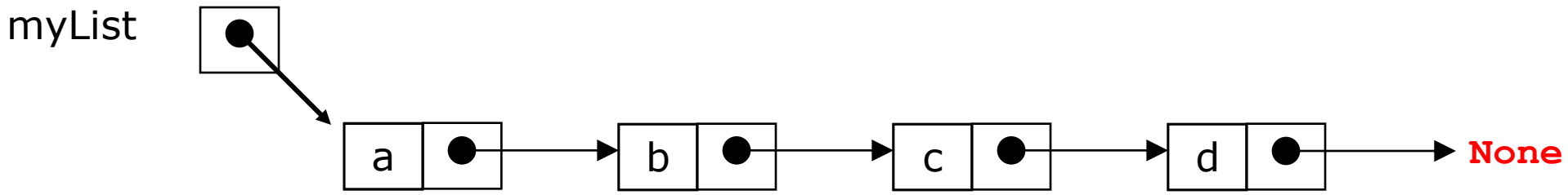
Data Structures & Algorithms in Python : Linked Lists

Dr. Owen Noel Newton Fernando

College of Computing and Data Science (CCDS)

Anatomy and terminology of linked lists

- A **`LinkedList`** consists of a connected sequence of **`Link/Node`** objects
- Each **`link/node`** contains a reference to a datum and a reference to the next **`node`** in the sequence.
  - The datum is just an object. It could be just an int or a string, or it could contain a key/data pair.
- The last node in the sequence refers to **`None`** as its next **`node`**
- The sequence of **`nodes`** is accessed via a "header" object that contains a reference to the first **`node`** in the sequence.



Content Copyright Nanyang Technological University

More terminology

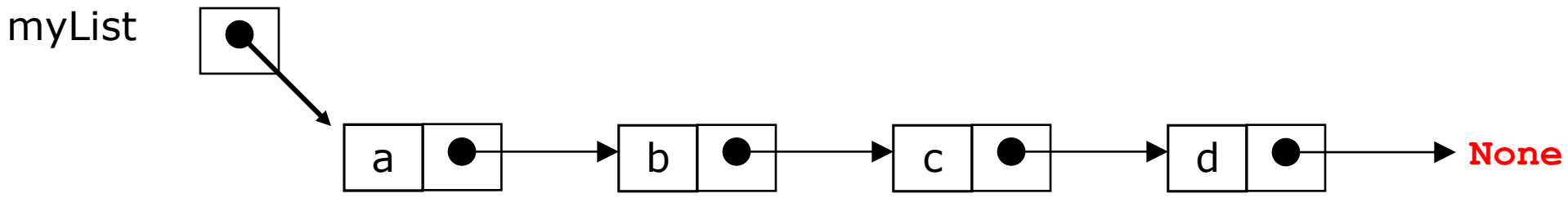
- A node's successor is the next node in the sequence
  - The last Link has no successor
- A node's predecessor is the previous node in the sequence
  - The first Link has no predecessor
- A LinkedList's length is the number of nodes in it
  - A linked list may be empty (contain no nodes)

Content Copyright Nanyang Technological University

Real Life Scenario

Singly-linked lists

- Here is a basic singly-linked LinkedList:



- Each Node contains a datum and a field called **`__next`** with a reference to its successor (the last Node has no successor)

- The header points to the first Node in the list (or contains a reference to **`None`** if the list is empty)

Content Copyright Nanyang Technological University

Building a linked list

- Creating Lists using,
  - Python lists (square brackets),
  - Classes (LinkedList class).

Content Copyright Nanyang Technological University

Python Lists

What is a Python List?

- A list in Python is a collection of items that can store multiple values in a single variable.

- **Key Points**:
  - Lists are mutable (changeable).
  - They can store mixed data types (integers, strings, etc.).
  - Lists are created using square brackets [ ]

Content Copyright Nanyang Technological University

Python List

```
# Creating a list of numbers
numbers = [1, 2, 4, 5]

# Creating a list with mixed data types
mixed_list = [1, "apple", 3.5, True]


# Access the first element (index 0)
first_element = numbers[0]
print(first_element)  # Output: 1

# Access the second element (index 1)
second_element = numbers[1]
print(second_element)  # Output: 2
```

Content Copyright Nanyang Technological University

Python List Operations

- **Methods to Add Elements**:
  - **append()**: Adds an element at the end of the list.
  - **insert()**: Adds an element at a specific index.

```
# Append an element to the list
numbers.append(6)
print(numbers)  # Output: [1, 2, 4, 5, 6]
```

1
2
4
5
6

```
# Insert an element at a specific
position
numbers.insert(2, 30)
print(numbers)  # Output: [1, 2, 30, 4,
5]
```

1
2
30
5

Content Copyright Nanyang Technological University

Python List Operations

- **Methods to Remove Elements**:
  - **remove():** Removes the first occurrence of an element.
  - **pop():** Removes and returns an element from a specific index (or the last item if no index is provided)

```
# Remove a specific element by value
numbers.remove(4) <----------------------------------
print(numbers)  # Output: [1, 2, 5]
```

1
2
4
5

```
# Remove an element by index using pop
removed_element = numbers.pop(1) <--------------------
print(removed_element)  # Output: 2
print(numbers)  # Output: [1, 4, 5]
```

1
2
4
5

Content Copyright Nanyang Technological University

Building a linked list cont.

- Creating Lists using,
  - Python lists (square brackets),
  - Classes (LinkedList class).

Content Copyright Nanyang Technological University

Using Classes

- Use two classes:
  - **`Link/Node`**, which contains:
    - Datum
    - Reference to the next **`Link/Node`**
  - **`LinkedList`**, which is the header for a list, and just holds a reference **`__first`** to the first **`Link`** in the list

Content Copyright Nanyang Technological University

Link / Node class

```
class Link(object):                # One element of a linked list
   def __init__(self, datum, next=None): # Constructor
      self.__data = datum          # The datum for this Link
      self.__next = next           # Reference to next Link

   def getData(self):          return self.__data    # Get the datum
   def setData(self, datum): self.__data = datum    # Set the datum
   def getNext(self): return self.__next            # Get the next reference
   def setNext(self, link):  self.__next = link     # Set the next reference

   def isLast(self):           return self.getNext() is None
   def __str__(self):          return str(self.getData())
```

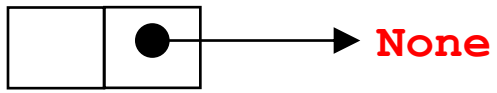
Content Copyright Nanyang Technological University

Link / Node class (code Explanation)

```
def __init__(self, datum, next=None): # Constructor
    self.__data = datum        # The datum for this Link
    self.__next = next         # Reference to next Link
```

Constructor (__init__ method):

The constructor initializes a new Link object.



Parameters:

- **datum**: This is the data (or value) stored in the node. It can be any object (an integer, string, or custom object).
- **next**: This is a reference to the next Link object in the linked list. If no next node is provided, it defaults to None, meaning this node is the last in the chain.

Content Copyright Nanyang Technological University

Link / Node class (code Explanation)

```
def getData(self):          return self.__data     # Get the datum
def setData(self, datum): self.__data = datum    # Set the datum
```

Get and Set Data:

getData(): This method returns the data stored in the node. It gives access to the value without directly accessing private variables.

setData(): This method allows updating the data stored in the node. It replaces the current value with the new datum.

Content Copyright Nanyang Technological University

Link / Node class (code Explanation)

```
def getNext(self): return self.__next          # Get the next reference
def setNext(self, link):  self.__next = link    # Set the next reference
```

Get and Set Next Reference:

getNext(): This method returns the reference to the next node in the list. If it's the last node, it will return None.

setNext(): This method sets the reference to the next node. It links the current node to the link (which is another Link object), forming the connection in the linked list.

Content Copyright Nanyang Technological University

Link / Node class (code Explanation)

```
def isLast(self):          return self.getNext() is None
```

Check if this is the Last Node:

None: In Python, None is a special constant that represents the absence of a value or a null reference.

is: The is operator checks for identity, meaning it checks if two references point to the same object in memory.

The is None part checks if the next reference is None, which is a common way of identifying the end of a linked list.

So, this method returns True if the current node is the last node (because its next reference is None), and False otherwise.

Content Copyright Nanyang Technological University

Link / Node class (code Explanation)

```
def __str__(self):        return str(self.getData())
```

__str__ (Special Method):

This special method is automatically called when you try to print an object. It defines how the object should be represented as a string.

When you print a Link object, __str__() will return the string representation of the node's data.

Content Copyright Nanyang Technological University

The LinkedList class

- The most important thing in the **`LinkedList`** class is a reference to the first **`Link`** in the list:

```
class LinkedList(object):
    def __init__(self):  self.__first = None
```

- And some of the methods:
  - **`isEmpty`** - returns True if the LinkedList is empty
  - **`__str__`** - returns a string representation of the LinkedList
  - **`insert`** - inserts a Link at start of list
  - **`find`** - finds a Link with a particular key in the list
  - **`delete`** - deletes a Link with a particular key in the list

Content Copyright Nanyang Technological University

The LinkedList class

```
class LinkedList(object):       # A linked list of data elements
   def __init__(self):          # Constructor
      self.__first = None       # Reference to first Link

   def isEmpty(self): return self.getFirst() is None

   def insert(self, datum):     # Insert a new datum at start of list
      link = Link(datum, self.getFirst()) # Make a new Link containing datum
      self.setFirst(link)       # Update reference to front of LinkedList

   # return the Link associated with the specified goal key
   # Client furnishes a function that extracts a key from the datum
   def find(self, goal, key=identity):
      link = self.getFirst()                # Start at first link
      while link is not None:               # Search until the end of the list
         if key(link.getData()) == goal:    # Does this Link match?
            return link                     # If so, return the Link itself
         link = link.getNext()              # Else, continue on along list
```

Content Copyright Nanyang Technological University

The LinkedList class (code Explanation)

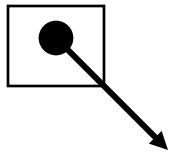
```
def __init__(self):          # Constructor
    self.__first = None      # Reference to first Link
```

Constructor (__init__ method):

__init__: This is the constructor that initializes an empty linked list. The attribute __first is set to None, meaning the list starts empty and has no nodes.

__first: This is a private variable that holds a reference to the first node in the linked list. When __first is None, it means the list is empty.

myList



Content Copyright Nanyang Technological University

The LinkedList class (code Explanation)

```
def isEmpty(self): return self.getFirst() is None
```

isEmpty Method:

This method checks whether the linked list is empty.

Logic: It returns True if the reference to the first node **(self.getFirst()) is None** (i.e., the list is empty), and False otherwise.

Content Copyright Nanyang Technological University

The LinkedList class (code Explanation)

```
def insert(self, datum):     # Insert a new datum at start of list
    link = Link(datum, self.getFirst()) # Make a new Link containing datum
    self.setFirst(link)      # Update reference to front of LinkedList
```

insert Method:

This method inserts a new element (or datum) at the start of the linked list.

Process:

It creates a new Link object that holds the provided datum (data) and the current first node (returned by self.getFirst()).

The new Link becomes the first node in the list by updating the reference self.setFirst(link).

Content Copyright Nanyang Technological University

Inserting a Link into a LinkedList

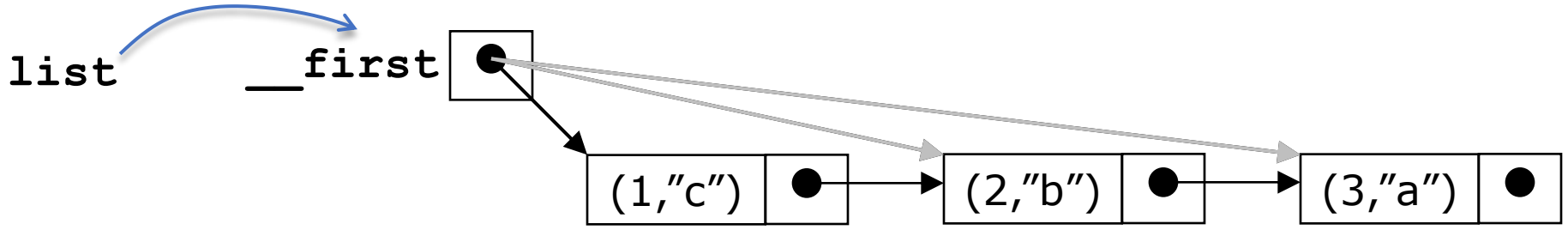
- There are many ways you might want to insert a new link into a list:
  - As the new first element (already seen - This is probably the easiest method to implement)
  - As the new last element
  - Before a link containing a given key
  - After a link containing a given key
- All are possible, but differ in difficulty

Content Copyright Nanyang Technological University

Creating a simple list and Inserting a new first element (animation)

- To create the list ((1,”c”), (2,”b”), (3,”a”)):

```
list = LinkedList()
list.insert((3, "a”))
list.insert((2, ”b”))
list.insert((1, "c”))
```



Content Copyright Nanyang Technological University

The LinkedList class (code Explanation)

```
# return the Link associated with the specified goal key
# Client furnishes a function that extracts a key from the datum
def find(self, goal, key=identity):
    link = self.getFirst()                # Start at first link
    while link is not None:               # Search until the end of the list
        if key(link.getData()) == goal:   # Does this Link match?
            return link                   # If so, return the Link itself
        link = link.getNext()             # Else, continue on along list
```

find Method:

This method searches the linked list to find the first node (or Link) whose data matches the provided goal.

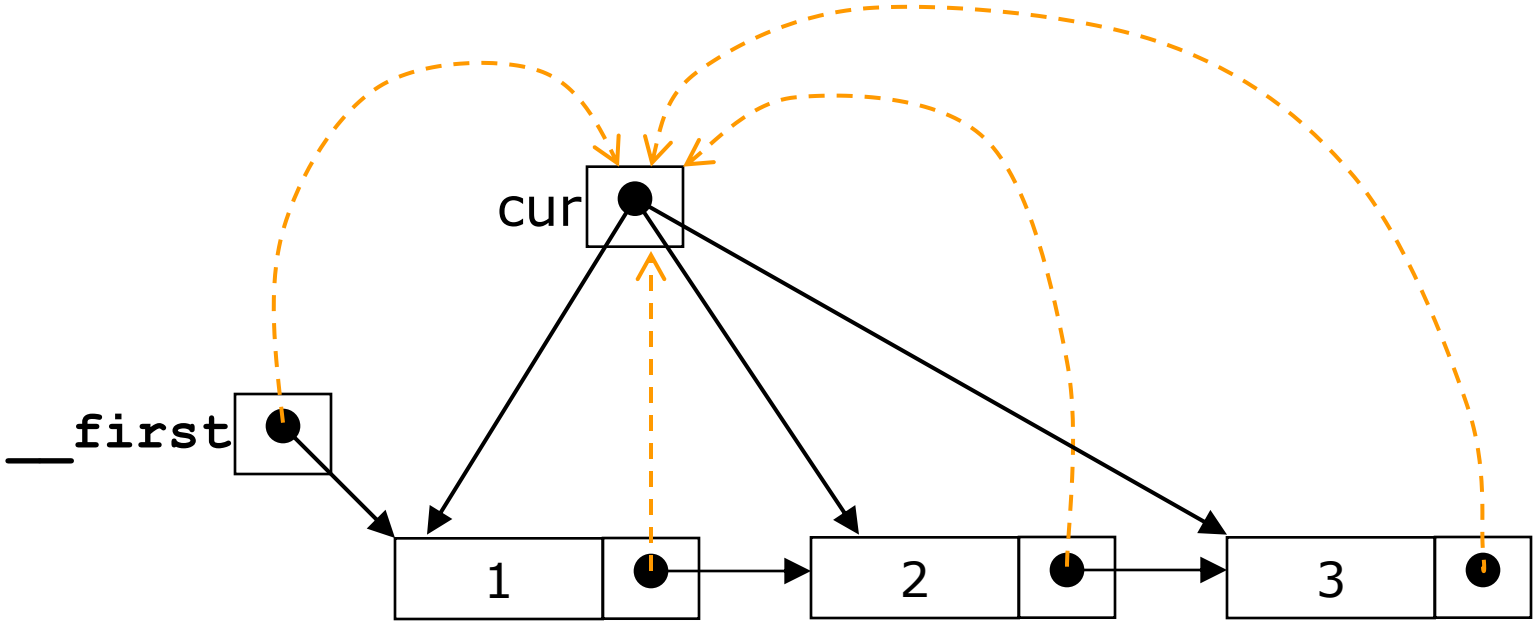
Parameters:

goal: The value being searched for in the list.

key: A function that extracts the comparison key from the data. By default, it's set to identity (i.e., it returns the data itself). This allows flexibility if the data is a more complex structure, such as a dictionary or an object, where you might want to extract a specific field for comparison.

Content Copyright Nanyang Technological University

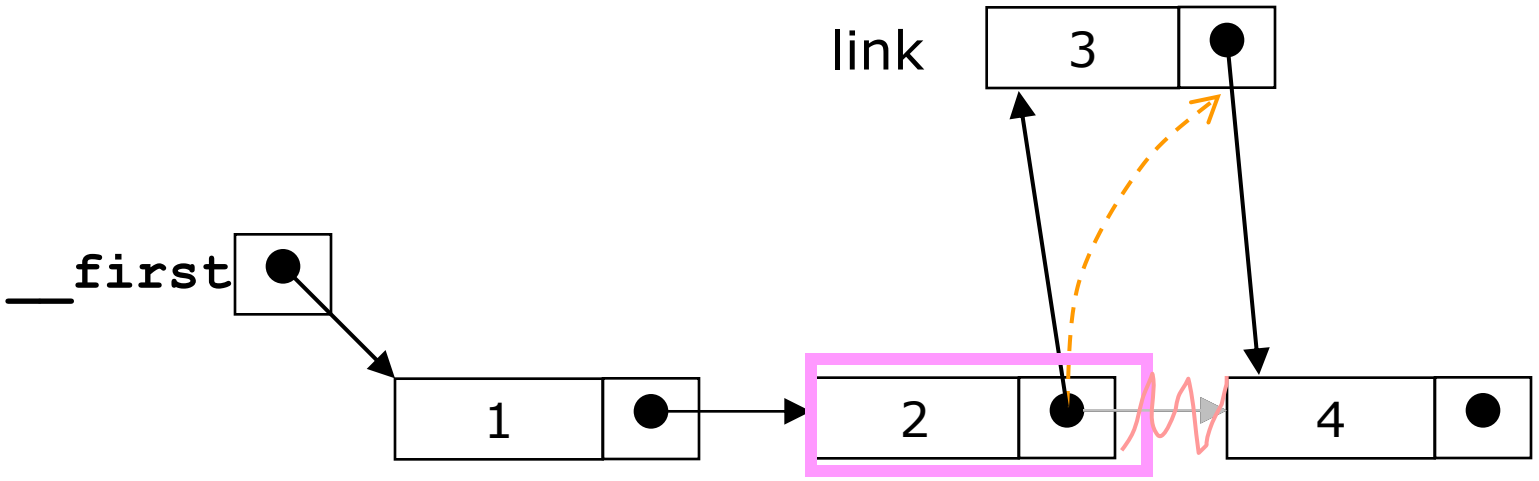
Traversing a LinkedList in find method (animation)




```
link = self.getFirst()
while link is not None:
    if key(link.getData()) == goal:
        return link
    link = link.getNext()
```

Content Copyright Nanyang Technological University

Insert cont. – Inserting after (animation)



Find the link you want to insert after

First, copy the pointer from the link that's already in the list

Then, change the pointer in the link that's already in the list

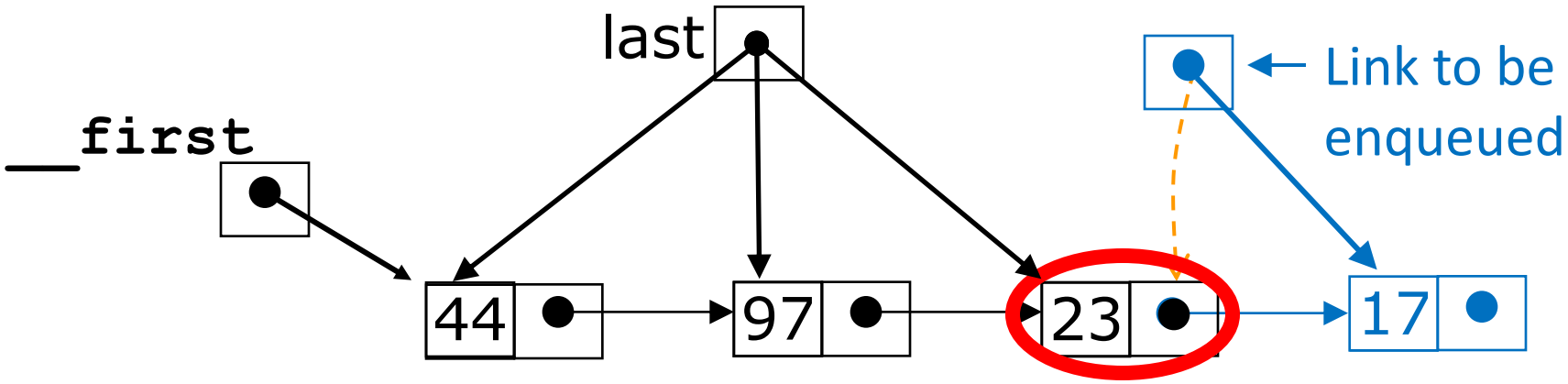
Content Copyright Nanyang Technological University

Inserting a Link after Link containing a given key

```
# Insert a new datum after the first Link with a matching key
def insertAfter(self, goal, newDatum, key=identity):
    link = self.find(goal, key)                # Find matching Link object
    if link is None: return False              # Can't find matching Link
    newLink = Link(newDatum, link.getNext())   # build a new Link node
    link.setNext(newLink)                      # and patch into list
    return True
```

Content Copyright Nanyang Technological University

Insert a Link at the end



To enqueue (add) a Link to the end:

* Find the current last Link by looping through the list
* Change its `__next` field to point to the new last Link

Special case when first == None

Content Copyright Nanyang Technological University

Converting a LinkedList to a string

```
def __str__(self):              # Build a string representation
    result = "["                # Enclose list in square brackets
    link = self.getFirst()      # Start with first link
    while link is not None:     # Keep going until no more links
        if len(result) > 1:     # After first link,
            result += " > "     # separate links with right arrowhead
        result += str(link)     # Append string version of link
        link = link.getNext()   # Move on to next link
    return result + "]"         # Close with square bracket
```

Content Copyright Nanyang Technological University

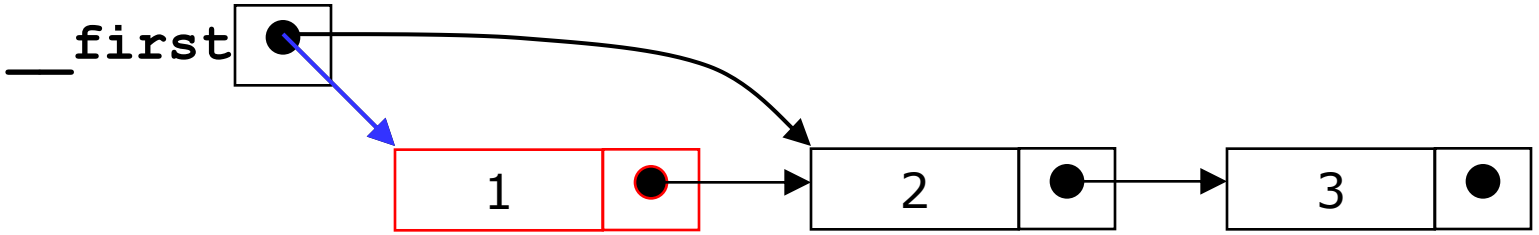
Deleting a Link from a LinkedList

- In order to delete a Link from a LinkedList, you have to change the pointer in its *predecessor*
- This is slightly tricky, because you can't follow a pointer backwards
- Deleting the first Link in a list is a special case, because the Link's predecessor is the list header

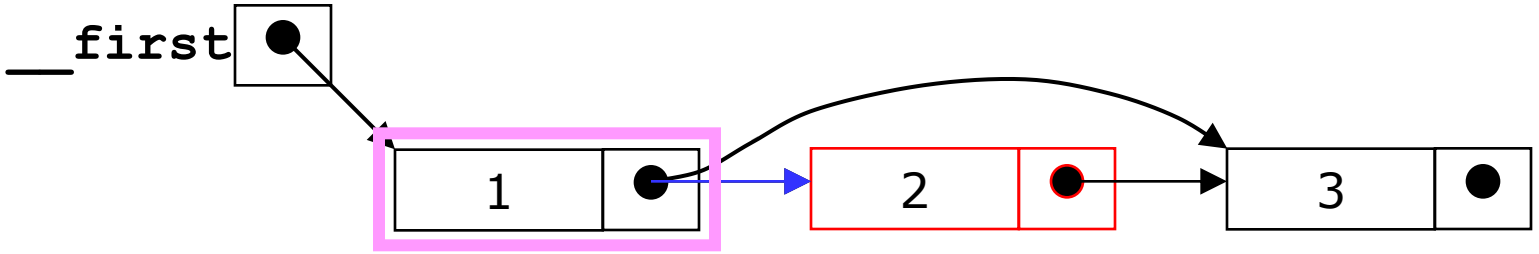
Content Copyright Nanyang Technological University

Deleting a Link from a LinkedList

- To delete the first element, change the link in the header



- To delete some other element, change the link in its predecessor



- Deleted Links will eventually be garbage collected

Content Copyright Nanyang Technological University

Deleting from a LinkedList

```
# Delete the first Link from the list whose key matches the goal
def delete(self, goal, key=identity):
    if self.isEmpty():        # Empty list? Raise an exception
        raise Exception("Cannot delete from empty linked list")

    previous = self                          # Link or LinkedList before Link
    while previous.getNext() is not None:    # to be deleted
        link = previous.getNext()            # Next link after previous
        if goal == key(link.getData()):      # If next Link matches,
            previous.setNext(                # change the previous' next
                link.getNext())              # to be Link's next and return
            return link.getData()            # data since match was found
        previous = link                      # Advance previous to next Link

    # Since loop ended without finding item, raise exception
    raise Exception("No item with matching key found in list")
```

Content Copyright Nanyang Technological University

Real world Application – Task Scheduling

Task Scheduling :

Linked lists are often used in real-world task scheduling, especially in operating systems and other environments where efficient management of tasks or processes is critical.

Advantages of Linked Lists in Task Scheduling

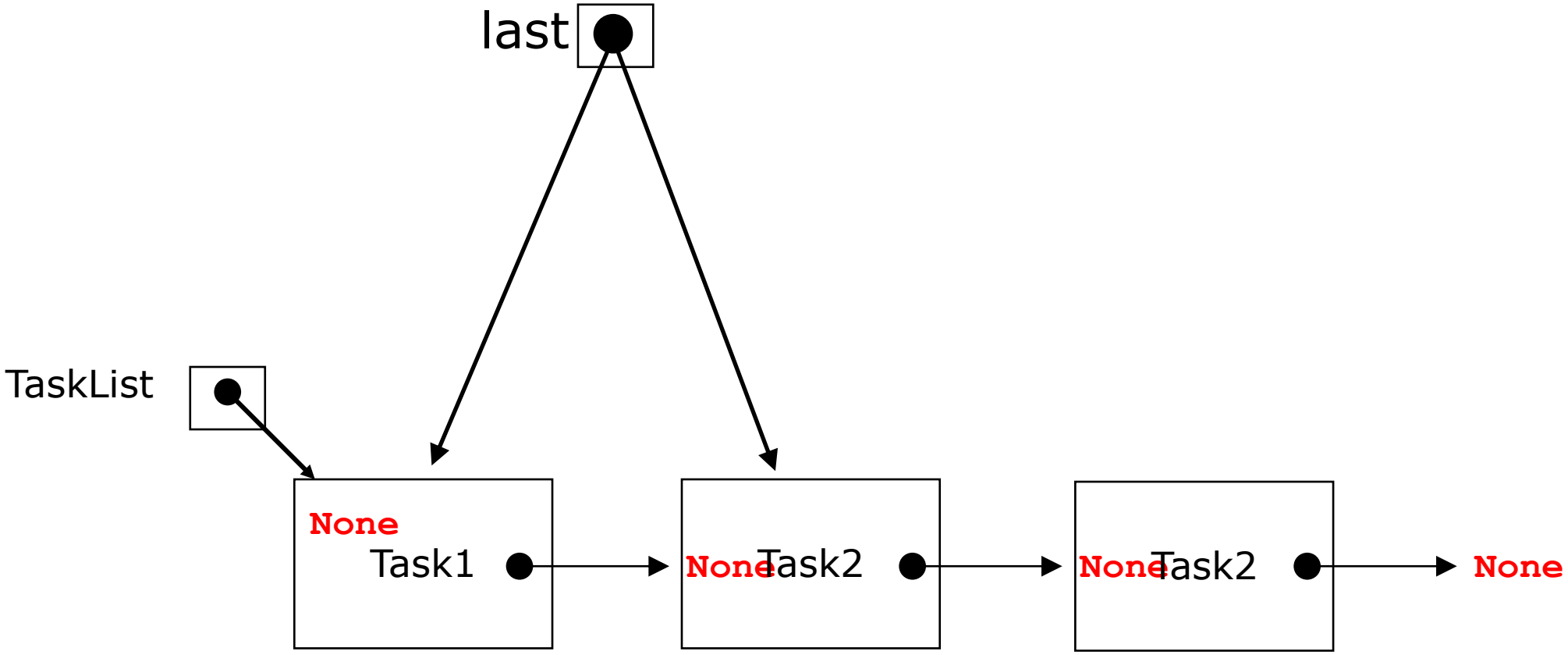
Efficient Insertion/Deletion: Linked lists allow for quick insertion and deletion of tasks, which is essential in environments where tasks frequently enter and exit the queue.

Dynamic Sizing: No pre-allocation of memory is needed, making linked lists ideal for systems with fluctuating task loads.

Content Copyright Nanyang Technological University

Real world Application – Task Scheduling

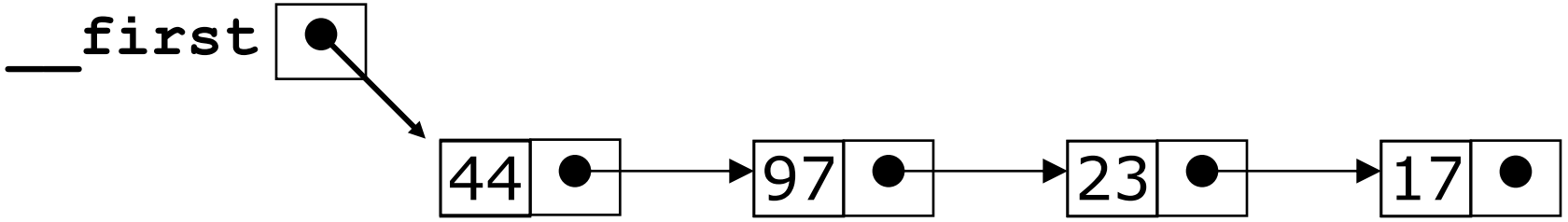
Inserting Task at the end



Content Copyright Nanyang Technological University

Can you implement a Stack with a LinkedList?

- Since all the action happens at the top of a stack, a LinkedList is a fine way to implement it
- The header of the list points to the top of the stack



- Pushing is inserting an element at the front of the list
- Popping is removing an element from the front of the list

Content Copyright Nanyang Technological University

Linked-list implementation details

- With a linked-list representation, overflow will not happen (unless you exhaust memory, which is another kind of problem)
- Underflow can happen, and should be handled the same way as for an array implementation
- When a Link's datum is popped from a list implementation of a stack, and the Link references an object, the reference (the pointer in the Link) does *not* need to be set to None
  - Unlike an array implementation, it really *is* removed—the garbage collector can no longer get to it from the linked list
  - Hence, garbage collection will occur as appropriate

Content Copyright Nanyang Technological University

Can we implement Queues with LinkedLists?

- In a queue, insertions occur at one end, deletions at the other end
- Operations at the front of a LinkedList are O(1), but at the other end they are O(n)
  - Why?
- BUT: there is a simple way to use a LinkedList list to implement both insertions at the front and deletions at the end in O(1) time
  - You always need a pointer to the first thing in the list
  - You can also keep an additional pointer to the *last* Link in the list
  - This is called a **double-ended** list

Content Copyright Nanyang Technological University

Queue implementation comparison

- With an array implementation:
  - you can have both overflow and underflow
  - you should set deleted elements to None

- With a linked-list implementation:
  - You can have underflow
  - Overflow is a global out-of-memory condition
  - There is no reason to set deleted elements to None

Content Copyright Nanyang Technological University